

MODULE 5: BASIC PROCESSING UNIT

SOME FUNDAMENTAL CONCEPTS

- To execute an instruction, processor has to perform following 3 steps:
 - 1) Fetch contents of memory-location pointed to by PC. Content of this location is an instruction to be executed. The instructions are loaded into IR, Symbolically, this operation is written as:
$$IR \leftarrow [PC]$$
 - 2) Increment PC by 4.
$$PC \leftarrow [PC] + 4$$
 - 3) Carry out the actions specified by instruction (in the IR).
- The first 2 steps are referred to as **Fetch Phase**.
Step 3 is referred to as **Execution Phase**.
- The operation specified by an instruction can be carried out by performing one or more of the following actions:
 - 1) Read the contents of a given memory-location and load them into a register.
 - 2) Read data from one or more registers.
 - 3) Perform an arithmetic or logic operation and place the result into a register.
 - 4) Store data from a register into a given memory-location.
- The hardware-components needed to perform these actions are shown in Figure 5.1.

COMPUTER ORGANIZATION

SINGLE BUS ORGANIZATION

- ALU and all the registers are interconnected via a **Single Common Bus** (Figure 7.1).
- Data & address lines of the external memory-bus is connected to the internal processor-bus via MDR & MAR respectively. (MDR → Memory Data Register, MAR → Memory Address Register).
- **MDR** has 2 inputs and 2 outputs. Data may be loaded
 - into MDR either from memory-bus (external) or
 - from processor-bus (internal).
- **MAR**'s input is connected to internal-bus;
MAR's output is connected to external-bus.
- **Instruction Decoder & Control Unit** is responsible for
 - issuing the control-signals to all the units inside the processor.
 - implementing the actions specified by the instruction (loaded in the IR).
- Register R0 through R(n-1) are the **Processor Registers**.
The programmer can access these registers for general-purpose use.
- Only processor can access 3 registers **Y, Z & Temp** for temporary storage during program-execution.
The programmer cannot access these 3 registers.
- In **ALU**,
 - 1) „A“ input gets the operand from the output of the multiplexer (MUX).
 - 2) „B“ input gets the operand directly from the processor-bus.
- There are 2 options provided for „A“ input of the ALU.
- MUX is used to select one of the 2 inputs.
- **MUX** selects either
 - output of Y or
 - constant-value 4 (which is used to increment PC content).

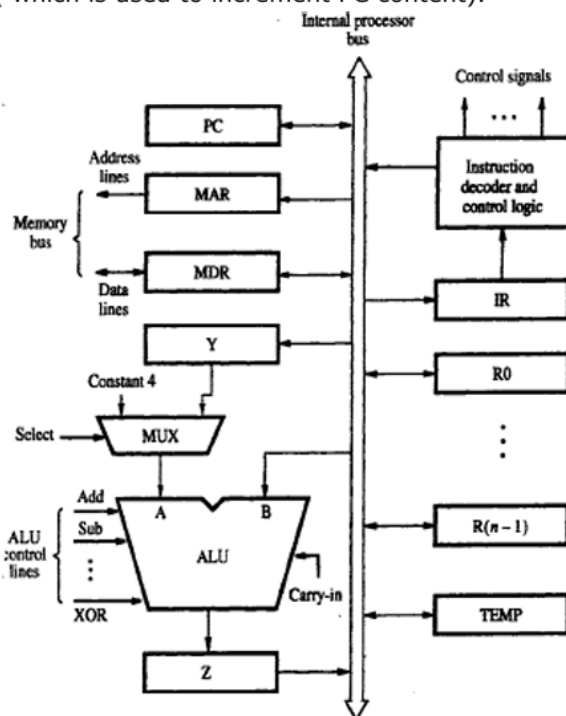


Figure 7.1 Single-bus organization of the datapath inside a processor.

- An instruction is executed by performing one or more of the following operations:
 - 1) Transfer a word of data from one register to another or to the ALU.
 - 2) Perform arithmetic or a logic operation and store the result in a register.
 - 3) Fetch the contents of a given memory-location and load them into a register.
 - 4) Store a word of data from a register into a given memory-location.

- **Disadvantage:** Only one data-word can be transferred over the bus in a clock cycle.

Solution: Provide multiple internal-paths. Multiple paths allow several data-transfers to take place in parallel.

COMPUTER ORGANIZATION

REGISTER TRANSFERS

- Instruction execution involves a sequence of steps in which data are transferred from one register to another.
- For each register, two control-signals are used: Ri_{in} & Ri_{out} . These are called **Gating Signals**.
- $Ri_{in}=1 \rightarrow$ data on bus is loaded into Ri . $Ri_{out}=1$
 $\rightarrow$ content of Ri is placed on bus.
 $Ri_{out}=0$, $\rightarrow$ bus can be used for transferring data from other registers.
- For example, *Move R1, R2*; This transfers the contents of register R1 to register R2. This can be accomplished as follows:
 - 1) Enable the output of registers R1 by setting $R1_{out}$ to 1 (Figure 7.2).
 This places the contents of R1 on processor-bus.
 - 2) Enable the input of register R2 by setting $R2_{in}$ to 1.
 This loads data from processor-bus into register R4.
- All operations and data transfers within the processor take place within time-periods defined by the **processor-clock**.
- The control-signals that govern a particular transfer are asserted at the start of the clock cycle.

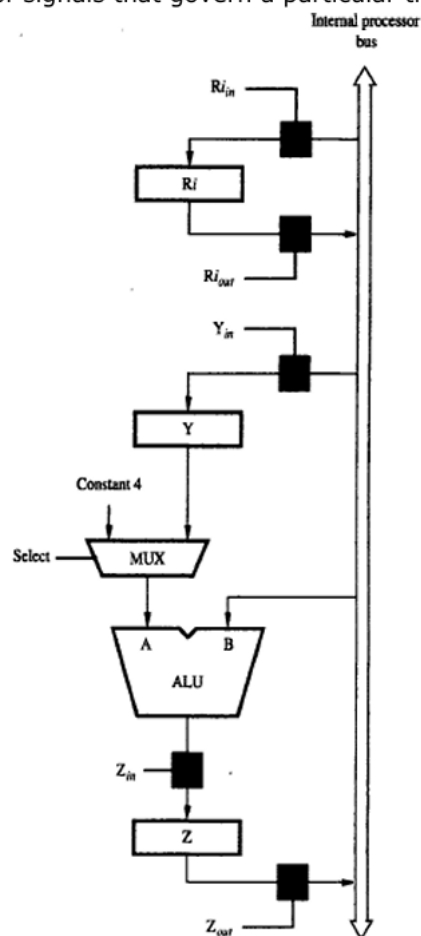


Figure 7.2 Input and output gating for the registers in Figure 7.1.

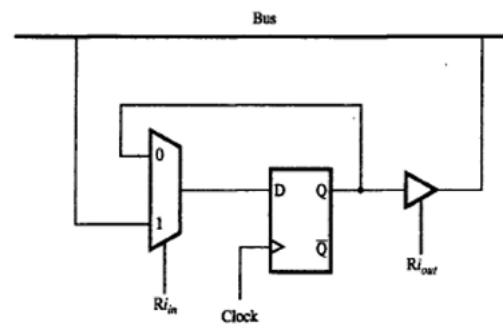


Figure 7.3 Input and output gating for one register bit.

Input & Output Gating for one Register Bit

- A 2-input multiplexer is used to select the data applied to the input of an edge-triggered D flip-flop.
- $Ri_{in}=1 \rightarrow$ mux selects data on bus. This data will be loaded into flip-flop at rising-edge of clock.
 $Ri_{in}=0 \rightarrow$ mux feeds back the value currently stored in flip-flop (Figure 7.3).
- Q output of flip-flop is connected to bus via a tri-state gate.
 $Ri_{out}=0 \rightarrow$ gate's output is in the high-impedance state.
 $Ri_{out}=1 \rightarrow$ the gate drives the bus to 0 or 1, depending on the value of Q.

COMPUTER ORGANIZATION

PERFORMING AN ARITHMETIC OR LOGIC OPERATION

- The ALU performs arithmetic operations on the 2 operands applied to its A and B inputs.
- One of the operands is output of MUX;
And, the other operand is obtained directly from processor-bus.
- The result (produced by the ALU) is stored temporarily in register Z.
- The sequence of operations for $[R3] \leftarrow [R1] + [R2]$ is as follows:
 - 1) $R1_{out}, Y_{in}$
 - 2) $R2_{out}, \text{SelectY, Add, } Z_{in}$
 - 3) $Z_{out}, R3_{in}$
- Instruction execution proceeds as follows:
 - Step 1 --> Contents from register R1 are loaded into register Y.
 - Step 2 --> Contents from Y and from register R2 are applied to the A and B inputs of ALU;
Addition is performed &
Result is stored in the Z register.
 - Step 3 --> The contents of Z register is stored in the R3 register.
- The signals are activated for the duration of the clock cycle corresponding to that step. All other signals are inactive.

CONTROL-SIGNALS OF MDR

- The MDR register has 4 control-signals (Figure 7.4):
 - 1) MDR_{in} & MDR_{out} control the connection to the internal processor data bus &
 - 2) MDR_{inE} & MDR_{outE} control the connection to the memory Data bus.
- MAR register has 2 control-signals.
 - 1) MAR_{in} controls the connection to the internal processor address bus &
 - 2) MAR_{out} controls the connection to the memory address bus.

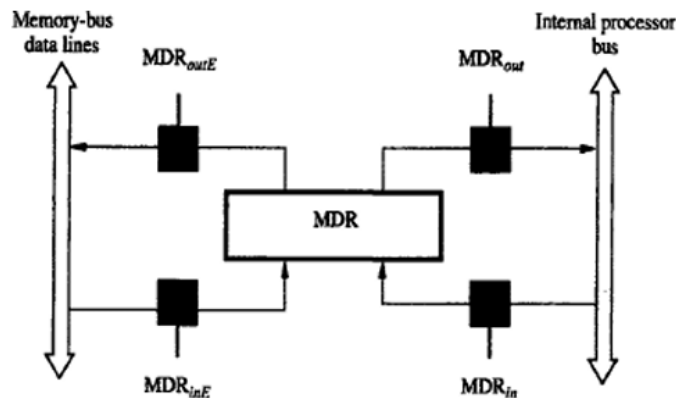


Figure 7.4 Connection and control signals for register MDR.

COMPUTER ORGANIZATION

FETCHING A WORD FROM MEMORY

- To fetch instruction/data from memory, processor transfers required address to MAR.
At the same time, processor issues Read signal on control-lines of memory-bus.
- When requested-data are received from memory, they are stored in MDR. From MDR, they are transferred to other registers.
- The response time of each memory access varies (based on cache miss, memory-mapped I/O). To accommodate this, MFC is used. (MFC → Memory Function Completed).
- MFC is a signal sent from addressed-device to the processor. MFC informs the processor that the requested operation has been completed by addressed-device.
- Consider the instruction Move (R1),R2. The sequence of steps is (Figure 7.5):
 - 1) $R1_{out}, MAR_{in}, Read$;desired address is loaded into MAR & Read command is issued.
 - 2) $MDR_{inE}, WMFC$;load MDR from memory-bus & Wait for MFC response from memory.
 - 3) $MDR_{outE}, R2_{in}$;load R2 from MDR.

where WMFC=control-signal that causes processor's control circuitry to wait for arrival of MFC signal.

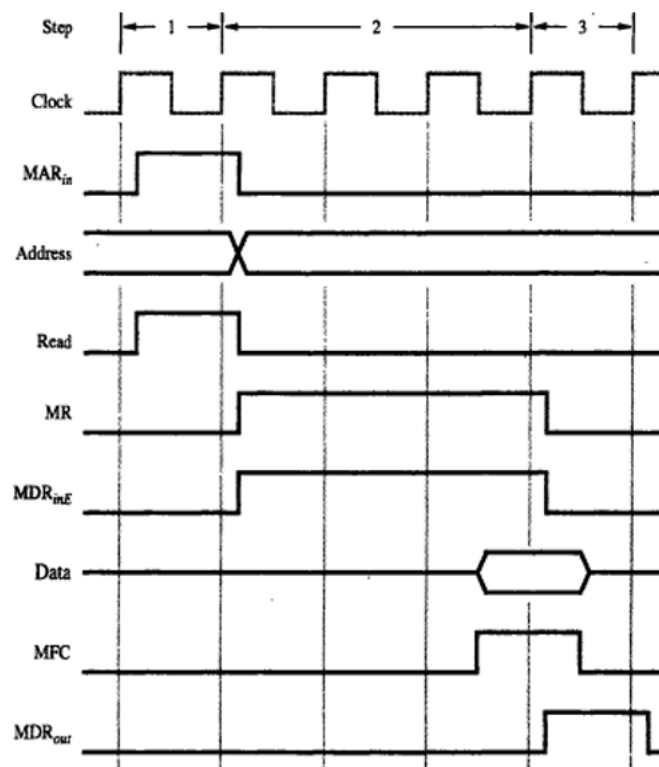


Figure 7.5 Timing of a memory Read operation.

Storing a Word in Memory

- Consider the instruction Move $R2,(R1)$. This requires the following sequence:
 - 1) $R1_{out}, MAR_{in}$;desired address is loaded into MAR.
 - 2) $R2_{out}, MDR_{in}, Write$;data to be written are loaded into MDR & Write command is issued.
 - 3) $MDR_{outE}, WMFC$;load data into memory-location pointed by R1 from MDR.

COMPUTER ORGANIZATION

EXECUTION OF A COMPLETE INSTRUCTION

• Consider the instruction *Add (R3),R1* which adds the contents of a memory-location pointed by R3 to register R1. Executing this instruction requires the following actions:

- 1) Fetch the instruction.
- 2) Fetch the first operand.
- 3) Perform the addition &
- 4) Load the result into R1.

Step	Action
1	$PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}$
2	$Z_{out}, PC_{in}, Y_{in}, WMFC$
3	MDR_{out}, IR_{in}
4	$R3_{out}, MAR_{in}, Read$
5	$R1_{out}, Y_{in}, WMFC$
6	$MDR_{out}, SelectY, Add, Z_{in}$
7	$Z_{out}, R1_{in}, End$

Figure 7.6 Control sequence for execution of the instruction *Add (R3),R1*

• Instruction execution proceeds as follows:

- Step1--> The instruction-fetch operation is initiated by
- loading contents of PC into MAR &
 - sending a Read request to memory.

The Select signal is set to Select4, which causes the Mux to select constant 4. This value is added to operand at input B (PC's content), and the result is stored in Z.

- Step2--> Updated value in Z is moved to PC. This completes the PC increment operation and PC will now point to next instruction.

- Step3--> Fetched instruction is moved into MDR and then to IR.

The step 1 through 3 constitutes the **Fetch Phase**.

At the beginning of step 4, the instruction decoder interprets the contents of the IR. This enables the control circuitry to activate the control-signals for steps 4 through 7.

The step 4 through 7 constitutes the **Execution Phase**.

- Step4--> Contents of R3 are loaded into MAR & a memory read signal is issued.

- Step5--> Contents of R1 are transferred to Y to prepare for addition.

- Step6--> When Read operation is completed, memory-operand is available in MDR, and the addition is performed.

- Step7--> Sum is stored in Z, then transferred to R1. The End signal causes a new instruction fetch cycle to begin by returning to step1.

COMPUTER ORGANIZATION

BRANCHING INSTRUCTIONS

- Control sequence for an **unconditional branch instruction** is as follows:

Step	Action
1	$PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}$
2	$Z_{out}, PC_{in}, Y_{in}, WMFC$
3	MDR_{out}, IR_{in}
4	$Offset-field-of-IR_{out}, Add, Z_{in}$
5	Z_{out}, PC_{in}, End

Figure 7.7 Control sequence for an unconditional Branch instruction.

- Instruction execution proceeds as follows:
 - Step 1-3--> The processing starts & the fetch phase ends in step3.
 - Step 4--> The offset-value is extracted from IR by instruction-decoding circuit.
 - Since the updated value of PC is already available in register Y, the offset X is gated onto the bus, and an addition operation is performed.
 - Step 5--> the result, which is the branch-address, is loaded into the PC.
- The branch instruction loads the branch target address in PC so that PC will fetch the next instruction from the branch target address.
- The branch target address is usually obtained by adding the offset in the contents of PC.
- The offset X is usually the difference between the branch target-address and the address immediately following the branch instruction.
- In case of **conditional branch**,
 - we have to check the status of the condition-codes before loading a new value into the PC.
 - e.g.: $Offset-field-of-IR_{out}, Add, Z_{in}$, If $N=0$ then End
 - If $N=0$, processor returns to step 1 immediately after step 4.
 - If $N=1$, step 5 is performed to load a new value into PC.

Figure 7.8 Three-bus organization of the datapath.

HARDWIRED CONTROL

-

Figure 7.12 Generation of the Z_{in} control signal

Chapter 8. Pipelining

Instruction Pipelining – Basic Concepts

Need for Pipelining

- Program execution speed depends on:
 - Processor and memory speed
 - Hardware organization
- Performance can be improved by:
 - Using faster circuits
 - **Allowing multiple operations to occur simultaneously**
- Even if the time for a single operation does not change, **overall throughput (operations per second) increases** through concurrency.

. Concept of Concurrent Activity

- Concurrent activities were earlier introduced through:
 - **Multiprogramming**
 - **Overlapping I/O transfers and computation**
- **DMA (Direct Memory Access)** devices enable I/O operations to proceed independently once initiated by the processor.

. What is Pipelining?

- **Pipelining is an effective technique for organizing concurrent activity in a computer system.**
- It is analogous to an **assembly line in manufacturing**:
 - Each stage performs a specific task
 - Multiple items are processed simultaneously at different stages
- Example:
 - While one car is having its engine installed,
 - Another car is getting its body fitted,
 - A third car is having its chassis prepared
- Though one car takes a long time to complete, **cars are produced at regular short intervals.**

Pipelining Applied to Computers

- A processor executes a program by:
 - Fetching instructions
 - Executing instructions
- These steps normally occur **sequentially**:
 - Fetch instruction → Execute instruction → Fetch next → Execute next

Let:

- **Fi** = Fetch step for instruction *li*
- **Ei** = Execute step for instruction *li*

Execution consists of a **sequence of fetch and execute steps.**

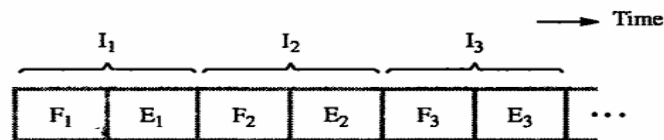
5. Two-Stage Pipeline Concept

- Consider a computer with **two separate hardware units**:
 - Fetch Unit**
 - Execution Unit**
- While:
 - The execution unit executes instruction I_i
 - The fetch unit simultaneously fetches instruction I_{i+1}

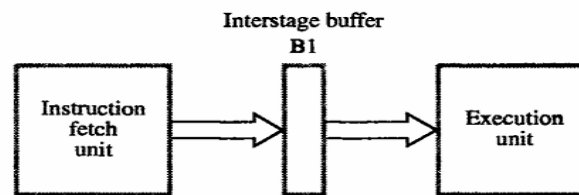
6. Role of Intermediate Buffer

- The fetched instruction is stored in an **intermediate buffer (B1)**.
- Purpose of buffer B1:
 - Allows the execution unit to work on the current instruction
 - Enables the fetch unit to fetch the next instruction simultaneously
- This buffer is essential for **overlapping fetch and execute operations**.

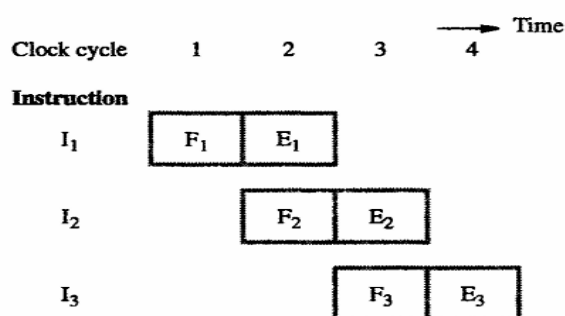
8.1 BASIC CONCEPTS



(a) Sequential execution



(b) Hardware organization



(c) Pipelined execution

Figure 8.1 Basic idea of instruction pipelining.

The computer is controlled by a clock whose period is such that the fetch and execute steps of any instruction can each be completed in one clock cycle. Operation of the computer proceeds as in Figure 8.1c. In the first clock cycle, the fetch unit fetches an instruction I_1 (step F_1) and stores it in buffer B1 at the end of the clock cycle. In the second clock cycle, the instruction fetch unit proceeds with the fetch operation for instruction I_2 (step F_2). Meanwhile, the execution unit performs the operation specified by instruction I_1 , which is available to it in buffer B1 (step E_1). By the end of the

Four stage Pipeline

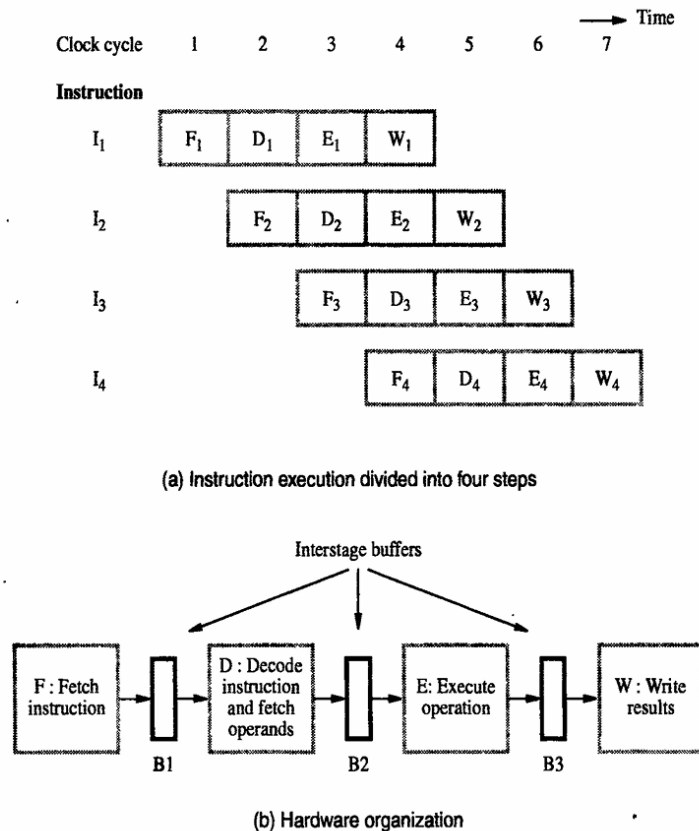


Figure 8.2 A 4-stage pipeline.

A pipelined processor processes each instruction in **four steps**, as listed below:

Pipeline Stages

- **F – Fetch:**
Read the instruction from memory.
- **D – Decode:**
Decode the instruction and fetch the required source operand(s).
- **E – Execute:**
Perform the operation specified by the instruction.
- **W – Write:**
Store the result in the destination location.

Pipeline Operation

- The sequence of events for this pipeline is illustrated in **Figure 8.2a**.
- At any given time, **four instructions are in progress simultaneously**.
- Therefore, **four distinct hardware units** are required, one for each pipeline stage.
- These hardware units must:
 - Operate **simultaneously**
 - Work **without interfering** with one another

Role of Storage Buffers

- Information flows from one pipeline stage to the next through **storage buffers**.
- As an instruction moves through the pipeline, **all information required by subsequent stages must be passed along**.

Buffer Contents During Clock Cycle 4 (Example)

- **Buffer B1**
 - Holds **instruction I₃**
 - Instruction I₃ was fetched in cycle 3
 - It is currently being decoded by the instruction-decoding unit
- **Buffer B2**
 - Holds:
 - Source operands for **instruction I₂**
 - Specification of the operation to be performed
 - This information is produced by the decoding hardware in cycle 3
 - Also contains information required for the **Write stage (W₂)** of instruction I₂
 - Even though this information is not needed in the Execute stage, it must be passed to stage W in the next cycle
- **Buffer B3**
 - Holds:
 - Results produced by the execution unit
 - Destination information for **instruction I₁**

8.1.2 Pipeline Performance –

. Instruction Throughput in a Pipelined Processor

- A pipelined processor can **complete one instruction per clock cycle** once the pipeline is full.
- Hence, the **instruction processing rate** becomes **four times higher** than sequential execution in a **four-stage pipeline**.
- The **potential performance improvement** from pipelining is **proportional to the number of pipeline stages**.

. Ideal vs Practical Pipeline Operation

- The theoretical performance gain is achieved **only if the pipeline operates continuously without interruption**.
- In practice, uninterrupted pipeline operation **cannot always be sustained** throughout program execution.

. Causes of Performance Degradation

- A pipeline stage may fail to complete its task within the **allocated clock cycle**.
- Example:
 - In a **four-stage pipeline**, the **Execute (E) stage** performs arithmetic and logic operations.
 - One clock cycle is usually sufficient for most operations.
 - However, some operations (e.g., **division**) require **multiple clock cycles**.

. Example of Multi-Cycle Execution

- An instruction I_2 requires **three clock cycles** (cycles 4 to 6) to complete execution.
- During this period:
 - The **Write stage** has no data to process and must remain idle (cycles 5 and 6).
 - The contents of **buffer B2** must remain unchanged until execution completes.

. Pipeline Blocking (Stalling)

- Because buffer **B2 cannot be updated**, the preceding pipeline stages are affected:
 - Stage 2 cannot proceed
 - Stage 1 is also blocked
- This occurs because **buffer B1 cannot be overwritten**.
- As a result:
 - Instruction fetch and decode steps (D_4 and F_5) must be **postponed**
 - The pipeline experiences a **stall**

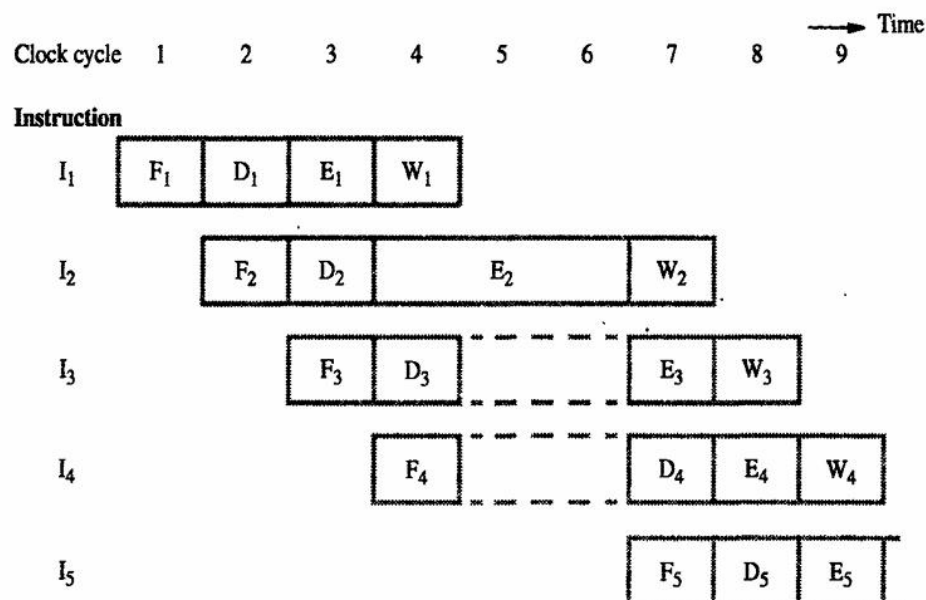


Figure 8.3 Effect of an execution operation taking more than one clock cycle.

- The pipelined operation shown in **Figure 8.3** is said to be **stalled for two clock cycles**.
- Normal pipelined operation **resumes in cycle 7**.

Pipeline Hazard

- Any condition that causes a pipeline to stall is called a hazard.

Data Hazard

- A **data hazard** occurs when:
 - The **source operands** or
 - The **destination operands**
- Are **not available at the time expected** in the pipeline.
- As a result:
 - Some operations must be **delayed**
 - The pipeline **stalls**

Instruction / Control Hazard

- The pipeline may also stall due to a **delay in the availability of an instruction**.
- One common cause is a **cache miss**, which requires the instruction to be fetched from **main memory**.
- Such hazards are often called:
 - **Control hazards**, or
 - **Instruction hazards**

Cache Miss Example

- The effect of a cache miss on pipelined operation is illustrated in **Figure 8.4**.
- **Instruction I₁:**
 - Is fetched from the cache in **cycle 1**
 - Executes normally
- **Instruction I₂:**
 - Fetch begins in **cycle 2**
 - Results in a **cache miss**
- Due to the cache miss:
 - The instruction fetch unit must **suspend further fetch requests**
 - It waits for instruction **I₂** to arrive from main memory
- Instruction **I₂** is:
 - Received and loaded into **buffer B1** at the **end of cycle 5**
- The pipeline then **resumes normal operation**.

- The previous pipeline is said to have been stalled for two clock cycles.
- Any condition that causes a pipeline to stall is called a hazard.
- Data hazard – any condition in which either the source or the destination operands of an instruction are not available at the time expected in the pipeline. So some operation has to be delayed, and the pipeline stalls.
- Instruction (control) hazard – a delay in the availability of an instruction causes the pipeline to stall.
- Structural hazard – the situation when two instructions require the use of a given hardware resource at the same time.